

Big Data Aplicado

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

Índice general

12. Monitorizacion: Herramientas	1
12.1. Ganglia	2
12.1.1. Arquitectura	2
12.2. Nagios Core	9
12.2.1. Arquitectura	10
12.2.2. Configuración	11
12.2.3. Integración con Ganglia	15

Listado de acrónimos

CSV	Comma Separated Values
JSON	JavaScript Object Notation
NSCA	Nagios Service Check Acceptor
NRPE	Nagios Remote Plugin Executor
RPC	Remote Procedure Call
RRA	Round Robin Arrays
RRD	Round Robin Database
URL	Uniform Resource Locator
XML	eXtensible Markup Language

Capítulo 12

Monitorizacion: Herramientas

El mejor sistema de monitorización para un clúster Hadoop es un sistema que se adapte al entorno y las necesidades de sus administradores y usuarios. Tanto la monitorización del uso de los recursos del sistema como la notificación de alertas inmediata si se producen un cambio repentino en el uso de los mismos, serán labores imprescindibles para estos sistemas. A su vez también podrán ser utilizados para asegurarse de que sólo los usuarios autorizados tienen acceso al sistema o supervisar las solicitudes de autenticación fallidas [OC14].

Las herramientas mas populares en monitorización de sistemas Hadoop son Ganglia ¹ y Nagios ², esto es debido a su flexibilidad y a que ofrecen gran variedad de medios para supervisar los recursos del sistema, las conexiones y cualquier otra parte del clúster Hadoop que sea posible que sea técnicamente posible monitorizar. Ambas son herramientas de código abierto con diferentes puntos fuertes que se complementan muy bien. Ganglia presenta una alta eficiencia en recopilación de métricas, el seguimiento de las mismas en el tiempo, y la agregación de los resultados, mientras que Nagios se centra más en proporcionar un mecanismo de alerta. Dado que la recopilación de métricas y las alertas son ambos aspectos esenciales de la supervisión, Ganglia y Nagios funcionan mejor juntos. Ambas herramientas tienen agentes que se ejecutan en todos los hosts de un clúster y recogen información a través de un proceso de sondeo (pooling) que puede sondear cualquiera de los hosts para obtener la información necesaria [Kuf16].

¹ Enlace: <http://http://ganglia.sourceforge.net>

² Enlace: <http://www.nagios.org>

12.1. Ganglia

Ganglia fue diseñado en la Universidad de California, Berkeley y comenzó como un proyecto de monitorización de código abierto destinado a ser utilizado con grandes sistemas distribuidos. La arquitectura abierta de Ganglia facilita la integración con otras aplicaciones y recopilar estadísticas sobre sus operaciones. Ganglia puede recibir y procesar datos de salida de Hadoop Metrics con facilidad y utilizarlos eficazmente [MLN⁺12].

Para un cluster monitorizado, cada *host* ejecuta un proceso *daemon* llamado *gmond* que recoge y difunde los datos locales de métricas (como el uso de la CPU, el uso de la memoria, etc.) a todos los hosts dentro del clúster. Un proceso de sondeo *gmetad* puede consultar cualquiera de los hosts, leer todos los datos de las métricas y enviarlos a un servidor central de monitorización. El host central puede mostrar las métricas, agregarlas o resumirlas para su uso posterior.

12.1.1. Arquitectura

A grandes rasgos, Ganglia tiene cuatro componentes principales:

- **gmond**: se ejecuta en todos los nodos recogiendo los datos de las métricas.
- **gmetad**: se encarga del sondeo de los datos de *gmond*.
- **rrdtool**: almacena los datos que se han sondeado mediante *gmetad*
- **gweb**: visualización y análisis de los datos almacenados.

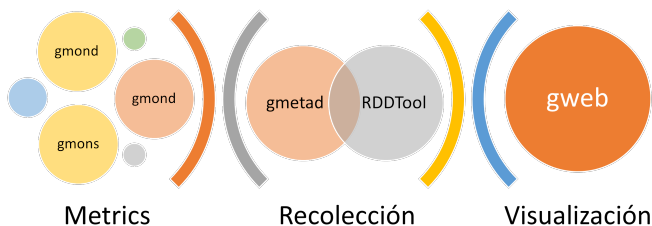


Figura 12.1: Visión esquemática de los agentes Ganglia

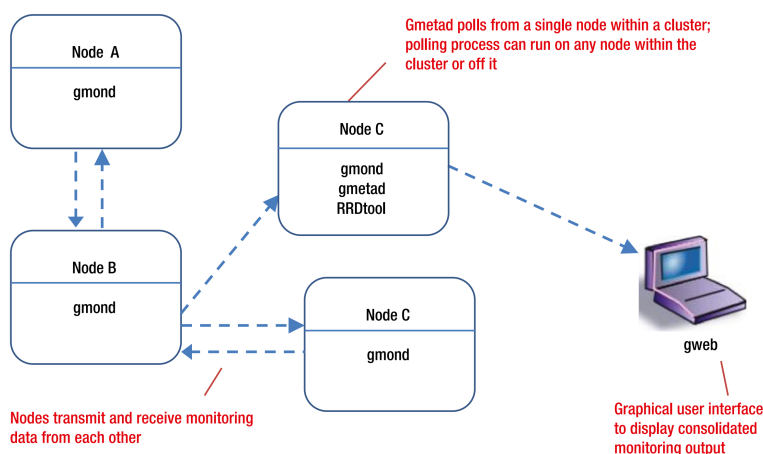


Figura 12.2: Arquitectura Ganglia para monitorizar Hadoop[Lak14]

gmond

gmond debe instalarse en cada uno de los hosts que desee supervisar. Interactúa con el sistema operativo del host para adquirir métricas de carga (por ejemplo, carga media del cluster), de proceso (por ejemplo, total de procesos en ejecución) o de transferencia (RPC) (por ejemplo, RpcAuthenticationFailures).

Es modular y utiliza *plugins* específicos del sistema operativo para realizar las mediciones. Dado que sólo se instalan los *plugins* necesarios en tiempo de compilación, gmond genera una sobrecarga insignificante.

gmond no se invoca a petición de un motor de sondeo externo (para la medición), sino que se realiza un sondeo de acuerdo con un calendario definido por un archivo de configuración local. Las mediciones son con otros hosts (del cluster) a través de un simple protocolo de escucha/anuncio emitido en la misma dirección de multidifusión.

Cada host de gmond también registra las métricas que recibe de otros *hosts* dentro del cluster. Por lo tanto, cada *host* en un cluster de Ganglia conoce el valor actual de cada métrica registrada por cada otro *host* en el mismo cluster. Esta es la razón por la que sólo es necesario sondear un *host* por clúster para obtener las métricas de todo el clúster, y cualquier fallo de un *host* individual no afectará al sistema en absoluto. Además, este diseño reduce exponencialmente el número de *hosts* que necesitan ser *encuestados*, y por lo tanto es fácilmente escalable para grandes clusters.

Listado 12.1: Ejemplo de configuración de gmod

```
1  globals {
2    daemonize = yes
3    setuid = yes
4    user = nobody
5    debug_level = 0
6    max_udp_msg_len = 1472
7    mute = no
8    deaf = no
9    allow_extra_data = yes
10   host_dmax = 0 /*secs */
11   host_tmax = 20 /*secs */
12   cleanup_threshold = 300 /*secs */
13   gexec = no
14   send_metadata_interval = 0 /*secs */
15 }
16
17 cluster {
18   name = "SR1"
19 }
20
21
22 udp_send_channel {
23   host = 192.168.0.8
24   port = 8649
25   ttl = 1
26 }
27
28 udp_rcv_channel {
29   port = 8649
30 }
31
32 tcp_accept_channel {
33   port = 8649
34 }
```

gmetad

gmetad es el proceso de sondeo dentro del sistema de monitorización de Ganglia. Necesita una lista de nombres de *host* que especifique al menos un host por cluster 12.2. Un volcado en formato XML de las métricas de un clúster puede ser solicitado por gmetad desde cualquier host del clúster en el puerto 8649, que es como gmetad obtiene los datos de las métricas de un cluster.

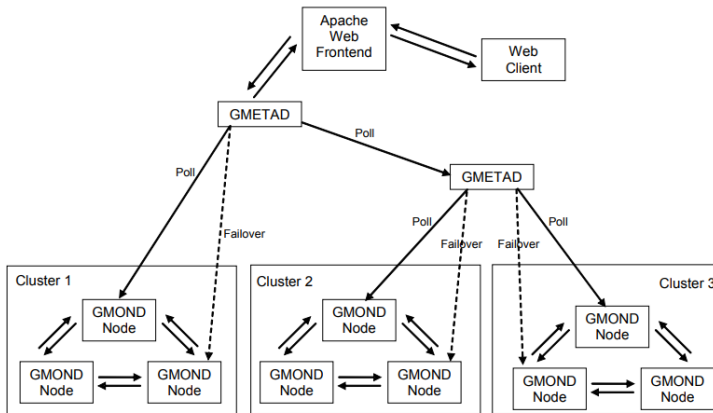


Figura 12.3: Rol de gmetad en la Arquitectura Ganglia

Listado 12.2: Ejemplo de configuración de gmetad

```

1  -- Format:
2  -- data_source "my cluster" [polling interval] address1:port addreses2:port ...
3
4  data_source "HDPTaskTracker" 50 localhost:8658
5  data_source "HDPDataNode" 50 localhost:8659
6  data_source "HDPNameNode" 50 localhost:8661
7  data_source "HDPJobTracker" 50 localhost:8662
8  data_source "HDPResourceManager" 50 localhost:8664
9  data_source "HDPHistoryServer" 50 localhost:8666
10
11 -- Round-Robin Archives
12
13 RRAs "RRA:AVERAGE:0.5:1:244" "RRA:AVERAGE:0.5:24:244" "RRA:AVERAGE:0.5:168:244"
14 "RRA:AVERAGE:0.5:672:244" "RRA:AVERAGE:0.5:5760:374"
15
16
17 -- The name of this Grid. All the data sources above will be wrapped in a GRID
18 -- tag with this name.
19 -- default: unspecified
20 gridname "HDP_GRID"
21 --
22 case_sensitive_hostnames 1

```

Listado 12.3: Ejemplo de ejecución de gmetad

```

1 [root@... www]# /usr/local/ganglia/sbin/gmetad -d 10
2 Going to run as user nobody
3 Sources are ...
4 Source: [SR2, step 15] has 1 sources
5     129.253.128.112
6 Source: [SR1, step 15] has 11 sources
7     192.168.0.23
8     192.168.0.26
9     192.168.0.17
10    192.168.0.44
11    192.168.0.6
12    192.168.0.7
13    192.168.0.10
14    192.168.0.9
15    192.168.0.3
16    192.23.1.22
17    192.168.0.8
18 xml listening on port 8651
19 interactive xml listening on port 8652
20 cleanup thread has been started
21 Data thread 1168066880 is monitoring [SR2] data source
22     129.253.128.112
23 Data thread 1178556736 is monitoring [SR1] data source
24     192.168.0.23
25     192.168.0.26
26     192.168.0.17
27     192.168.0.44
28     192.168.0.6
29     192.168.0.7
30     192.168.0.10
31     192.168.0.9
32     192.168.0.3
33     192.23.1.22
34     192.168.0.8
35 [SR1] is a 2.5 or later data stream
36 hash_create size = 1024
37 hash->size is 1031
38 hash_create size = 50
39 hash->size is 53
40 hash_create size = 50
41 hash->size is 53
42 Updating host my.working-host.com, metric load_one
43 [...]
44 Updating host my.working-host.com, metric pkts_out
45 Updating host my.working-host.com, metric swap_free
46 Writing Summary data for source SR1, metric disk_free
47 Writing Summary data for source SR1, metric bytes_out
48 [...]
49 =====
50
51 Only one RRD is generating in
52 /var/lib/ganglia/rrds/SR1/my.working-host.com for this cluster (and
53 one for SR2).

```

RRDTool

RRDtool es el componente de Ganglia que se utiliza para almacenar los datos de las métricas sondeadas por gmetad desde cualquiera de los hosts del cluster. Las métricas se almacenan en forma de round-robin", es decir, cuando cuando no queda espacio para almacenar nuevos valores, los antiguos se sobrescriben.

De acuerdo con los requisitos de retención de datos, RRDtool agrega y almacena los valores de datos. Esta forma de almacenamiento de datos permite analizar rápidamente los datos recientes, así como mantener años de años de datos históricos utilizando una pequeña cantidad de espacio en disco. Además, como todo el espacio de disco necesario se asigna de antemano, la planificación de la capacidad es muy sencilla.

Son los archivos RRD los que almacenan los datos de las métricas en gmetad. Un archivo RRD consta de 1 o más RRAs. El valor por defecto de los RRAs es:

```
RRAs "RRA:AVERAGE:0.5:1:5856" \N - RRA:AVERAGE:0.5:1:5856
      "RRA:AVERAGE:0.5:4:20160" \N - "RRA:AVERAGE:0.5:1:5856
      "RRA:AVERAGE:0.5:40:52704"
```

Hay un valor llamado step (paso) que es el intervalo más pequeño en el que se procesan las métricas. Por defecto es de 15 segundos. La configuración mostrada consiste en 3 arrays. La primera mantiene un valor cada paso para 5856 valores. Esto es $5856 \times 15s$, lo que es $5856 \times 15 / 3600 = 24,4$ horas. Así que durante algo más de un día, se dispondrá de 4 valores por minuto. El segundo array mantiene la media de 4 pasos, que es un valor por minuto, durante 20160 minutos, es decir 14 días. La tercera matriz mantiene un valor cada 40 pasos o 10 minutos, durante $52704 \times 10 / 60 / 24 = 366$ días. Por lo tanto, para un año las métricas se mantienen en un intervalo de 10 minutos.

Los ficheros RRD se almacenan en la siguiente ruta por defecto:

```
/var/lib/ganglia/rrds/<nombre del clúster>/<nombre del nodo>/
```

donde cada métrica se almacena en un único archivo RRD como bytes_in.rrd

Para obtener los datos de los archivos .rrd de los ganglios y guardarlos en un archivo .xml, se puede utilizar rrdtool.

```
rrdtool dump
  filename.rrd [filename.xml]
  [--header|-h {none,xsd,.dtd}]
  [--no-header]
  [--daemon address]
  > filename.xml
```

Por ejemplo:

```
sudo rrdtool dump bytes_out.rrd > data.xml
```

gweb

gweb es la interfaz de visualización de Ganglia. Proporciona acceso instantáneo a cualquier métrica de cualquier host en el clúster sin especificar ningún detalle de configuración.

Resume visualmente toda el *grid* mediante gráficos que combinan las métricas por clúster y proporciona despleables para obtener detalles adicionales.

gweb permite cambiar el período de tiempo en los gráficos, soporta la extracción de datos en varios formatos textuales (CSV, JSON, y más), y proporciona una interfaz de URL totalmente funcional para que pueda incrustar los gráficos necesarios en otros programas a través de URLs específicas. Además, gweb es un programa PHP, que se ejecuta bajo el servidor web Apache y suele instalarse en el mismo hardware físico que gmetad, ya que necesita acceder a las bases de datos RRD creadas por gmetad.

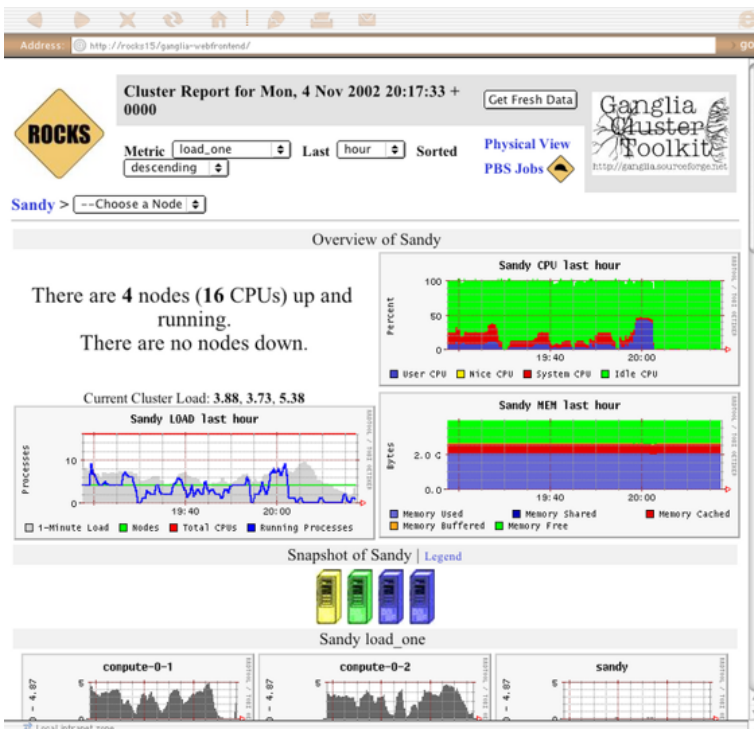


Figura 12.4: Ejemplo de interfaz Web de Gánlia

12.2. Nagios Core

Nagios Core es una aplicación de monitorización de sistemas y redes de código abierto. Monitoriza los hosts y servicios especificados alertando de las diferentes situaciones que se puedan presentar.

Algunas de características de Nagios Core incluyen:

- Monitorización de servicios de red (SMTP, POP3, HTTP, NNTP, PING, etc.)
- Monitorización de los recursos del host (carga del procesador, uso del disco, etc.)
- Diseño sencillo de plugins que permite a los usuarios desarrollar fácilmente sus propias comprobaciones de servicios
- Comprobaciones de servicio paralelas
- Posibilidad de definir la jerarquía de hosts de la red utilizando hosts "padres", lo que permite detectar y distinguir entre los hosts que están caídos y los que son inalcanzables
- Notificaciones de contacto cuando se producen problemas de servicio o de host y se resuelven (por correo electrónico, etc.)
- Posibilidad de definir manejadores de eventos que se ejecuten durante eventos de servicio o de host para la resolución proactiva de problemas
- Rotación automática de archivos de registro
- Soporte para implementar hosts de monitorización redundantes
- Interfaz web opcional para ver el estado actual de la red, el historial de notificaciones y problemas, el archivo de registro, etc.

Una de las principales características de Nagios es la flexibilidad. Todo el sistema de monitorización se basa en una serie de componentes u objetos que coordinados ofrecen la funcionalidad de Nagios [Koc14].

- **Comandos:** Son definiciones de cómo Nagios debe realizar determinados tipos de comprobaciones. Son una capa de abstracción sobre los plugins reales que permiten agrupar tipos de operaciones similares.
- **Períodos de tiempo:** Son intervalos de fecha y hora en los que una operación debería o no debe realizarse una operación. Por ejemplo, de lunes a viernes, de 09:00 a 17:00.
- **Hosts y grupos de hosts:** Son dispositivos junto con la posibilidad de agrupar hosts. Un mismo host puede ser miembro de más de un grupo.
- **Servicios:** Son diversas funcionalidades o recursos a supervisar en un host específico. Por ejemplo, el uso de la CPU, el espacio de almacenamiento o el servidor web.

- **Contactos y grupos de contactos:** Son personas a las que se debe notificar con información sobre cómo y cuándo deben ser contactados; los contactos pueden ser agrupados, y un mismo contacto puede ser miembro de más de un grupo.
- **Notificaciones:** Definen quién debe ser notificado de qué, por ejemplo, todos los por ejemplo, todos los errores del grupo linux-servers deben ir al grupo de contacto linux-admins linux-admins durante las horas de trabajo y al grupo de contacto critsit-team fuera de horas de trabajo. Las notificaciones no son estrictamente un objeto, sino una combinación de todos los objetos anteriores y son una parte esencial de Nagios.
- **Notificaciones escladas:** Son una extensión de las notificaciones; definen que después de un objeto está en el mismo estado durante un periodo de tiempo específico, otras personas deberían recibir notificar a otras personas de ciertos eventos-por ejemplo, un servidor crítico que esté fuera de servicio por más de 4 horas debe alertar a la administración de TI para que puedan rastrear el problema.

12.2.1. Arquitectura

El servicio de Nagios se ejecuta en un host y tiene plug-ins que se ejecutan en todos los hosts remotos que necesitan ser supervisados. Los *plug-ins* remotos de Nagios envían información y actualizaciones al servicio de Nagios, y la interfaz web de Nagios la muestra. Cuando se detectan problemas, el servicio de Nagios notifica a los contactos administrativos predefinidos mediante correo electrónico u otro medio predefinido. Los datos de registro históricos están disponibles en un archivo de registro definido en el archivo de configuración.

El sistema de monitorización de Nagios tiene tres componentes principales:

- **Servidor:** El servidor se encarga de gestionar y programar los complementos. A intervalos regulares el servidor comprueba el estado del plug-in y realiza una acción según el estado. En caso de alertas se notifican los recursos administrativos configurados.
- **Plug-ins:** Nagios proporciona un conjunto estándar de plug-ins configurables por el usuario, además de poder añadir más según sea necesario. Los plug-ins son programas ejecutables (en su mayoría escritos en C, Java, Python, etc.) que realizan una tarea específica y devuelven un resultado al servidor de Nagios.
- **Nagios Web:** Interfaz del navegador de Nagios: Son páginas web generadas por CGI que muestran información resumida información sobre los recursos monitorizados.

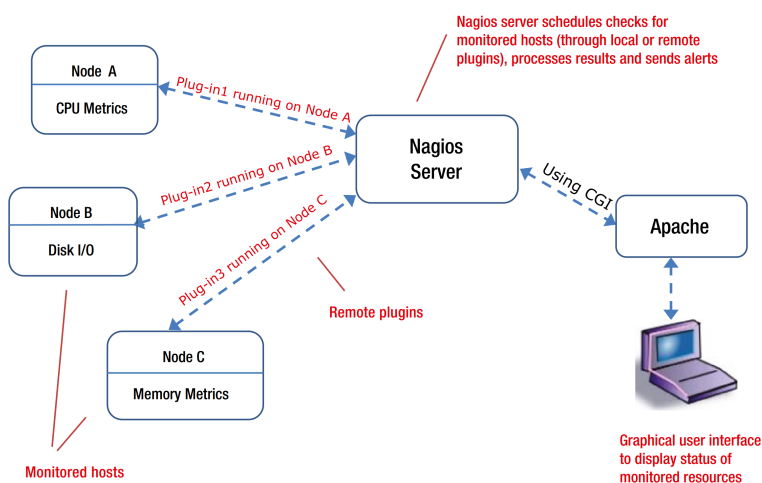


Figura 12.5: Arquitectura Nagios

12.2.2. Configuración

En Nagios Core existen varios archivos de configuración necesarios con el fin de monitorizar la infraestructura.

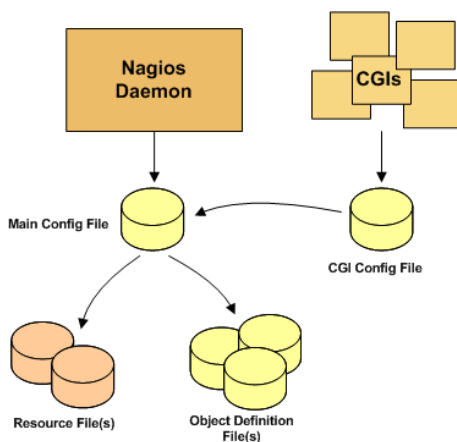


Figura 12.6: Archivos de Configuración de Nagios



Los archivos de configuración de ejemplo se instalan en el directorio `/usr/local/nagios/etc/` a partir de la guía de instalación rápida.

Archivo de Configuración Principal

El archivo de configuración principal contiene una serie de directivas que afectan al funcionamiento del *daemon* de Nagios Core. Este archivo de configuración es leído tanto por el *daemon* de Nagios Core como por los CGIs. Este archivo contiene la ubicación del archivo de registro de Nagios, el intervalo de actualización del estado de los hosts y servicios, el archivo de bloqueo y el archivo *status.dat*. Los usuarios y grupos de Nagios en los que se ejecutan las instancias se definen en este archivo. Tiene la ruta de todos los archivos de configuración de objetos individuales como comandos, contactos, plantillas, etc.

Listado 12.4: Ejemplo nagios.cfg

```
1 # LOG FILE
2 log_file=@localstatedir@/nagios.log
3
4 # OBJECT CONFIGURATION FILE(S)
5 # These are the object configuration files in which you define hosts,
6 # host groups, contacts, contact groups, services, etc.
7
8 # You can specify individual object config files as shown below:
9 cfg_file=@sysconfdir@/objects/commands.cfg
10 cfg_file=@sysconfdir@/objects/contacts.cfg
11 cfg_file=@sysconfdir@/objects/timeperiods.cfg
12 cfg_file=@sysconfdir@/objects/templates.cfg
13
14 # Definitions for monitoring the local (Linux) host
15 cfg_file=@sysconfdir@/objects/localhost.cfg
16
17 # OBJECT CACHE FILE
18 # This option determines where object definitions are cached when
19 # Nagios starts/restarts.
20 object_cache_file=@localstatedir@/objects.cache
21 precached_object_file=@localstatedir@/objects.precache
22
23 # RESOURCE FILE
24 # This is an optional resource file that contains USERx macro
25 # definitions.
26 resource_file=@sysconfdir@/resource.cfg
27
28 # STATUS FILE
29 # This is where the current status of all monitored services and
30 # hosts is stored. Its contents are read and processed by the CGIs.
31 status_file=@localstatedir@/status.dat
32
33 # STATUS FILE UPDATE INTERVAL
34 status_update_interval=10
35
36 # NAGIOS USER
37 nagios_user=@nagios_user@
38
39 # NAGIOS GROUP
40 nagios_group=@nagios_grp@
```

Archivo(s) de Recursos

Los archivos de recursos pueden utilizarse para almacenar macros definidas por el usuario *USERx* que pueden ser utilizadas en definiciones de comandos en los archivos de configuración del host.

El objetivo principal de tener archivos de recursos es utilizarlos para almacenar información de configuración sensible (como las contraseñas), sin ponerla a disposición de los CGIs.

También son útiles para especificar la ruta de los plugins y los manejadores de eventos. Si es necesario mover los plugins o los manejadores de eventos a un directorio diferente en el futuro, se puede simplemente actualizar una o dos macros *USERx*, en lugar de modificar las definiciones de los comandos. Los archivos de recursos también pueden utilizarse para almacenar directivas de configuración para fuentes de datos externas como MySQL.

Se puede especificar uno o más archivos de recursos opcionales utilizando la directiva `resource_file` en el archivo de configuración principal.

Listado 12.5: Ejemplo resource.cfg

```
1 # Sets USER1 to be the path to the plugins
2 $USER1$=@libexecdir@
3
4 # Sets USER2 to be the path to event handlers
5 $USER2=@libexecdir@/eventhandlers
6
7 # Store some usernames and passwords (hidden from the CGIs)
8 $USER3=someuser
9 $USER4=somepassword
```

Archivos de Definición de Objetos

Los archivos de definición de objetos se utilizan para definir hosts, servicios, grupos de hosts, contactos, grupos de contactos, comandos, etc. Aquí es donde se definen todas las elementos que se quieren monitorizar y cómo se quieren monitorizar.

Puede especificar uno o más archivos de definición de objetos utilizando las directivas `cfg_file` y/o `cfg_dir` en su archivo de configuración principal.

- **commands.cfg:** Este archivo de configuración proporciona algunos ejemplos de definiciones de comandos que puede consultar en las definiciones de hosts, servicios y contactos. Estos comandos se utilizan para comprobar y monitorizar hosts y servicios. Se puede ejecutar estos comandos localmente en una consola Linux donde también obtendrá la salida del comando que ejecute.

Listado 12.6: Ejemplo commands.cfg

```

1  define command {
2      command_name check_local_disk
3      command_line $USER1$/check_disk -w $ARG1$ -c $ARG2$ -p $ARG3$
4  }
5
6  define command {
7      command_name check_local_load
8      command_line $USER1$/check_load -w $ARG1$ -c $ARG2$
9  }
10
11 define command {
12     command_name check_local_procs
13     command_line $USER1$/check_procs -w $ARG1$ -c $ARG2$ -s $ARG3$
14 }

```

- **contacts.cfg:** Este archivo contiene información de contactos y grupos de Nagios. Por defecto, un contacto ya está presente Nagios admin.

```

define contact {
    contact_name nagiosadmin
    use generic-contact
    alias Nagios Admin
    email admin-nagios@nagios.com
}

define contactgroup {
    contactgroup_name admins
    alias Nagios Administrators
    members nagiosadmin
}

```

- **plantillas.cfg:** Este archivo de configuración le proporciona algunas plantillas de definición de objetos de ejemplo que son referidas por otras definiciones de host, servicio, contacto, etc. en otros archivos de configuración.
- **timeperiods.cfg:** Este archivo de configuración proporciona algunos ejemplos de definiciones de periodos de tiempo a los que se puede hacer referencia en las definiciones de host, servicio, contacto y dependencia.

Archivo de Configuración CGI

El fichero de configuración del CGI contiene una serie de directivas que afectan al funcionamiento de los CGIs. También contiene una referencia al archivo de configuración principal, para que los CGIs sepan cómo ha configurado Nagios y dónde están almacenadas sus definiciones de objetos.

Por defecto, el archivo de configuración de los CGI de Nagios se llama `cgi.cfg`.

Los CGIs leerán los archivos de configuración principal y de host para cualquier otro dato que puedan necesitar. Contiene toda la información de usuarios y grupos y sus derechos y permisos.

12.2.3. Integración con Ganglia

Los complementos remotos o locales se ejecutan y su estado es comparado por Nagios con estado especificado por el usuario/mapeo de notificaciones para realizar cualquier tarea de notificación necesaria. Servicios como NRPE NSCA se utilizan para las ejecuciones remotas. Si usted está [WZL13].

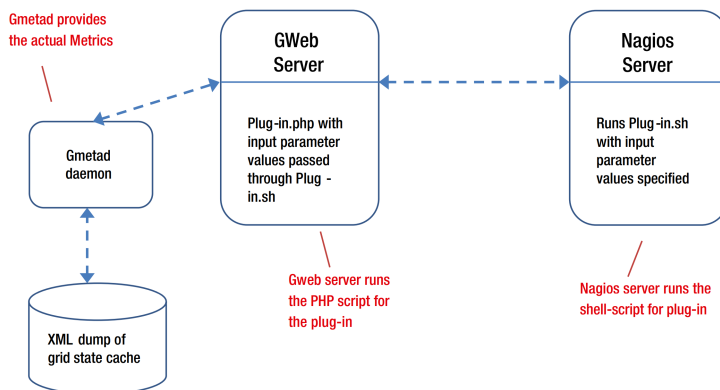


Figura 12.7: Integración Ganglia Nagios[Lak14]

Si se utiliza Ganglia para la monitorización todas las métricas que Nagios necesita (para CPU, memoria, E/S de disco, etc.), Ganglia las hace disponibles simplemente integrando las dos herramientas. Ganglia incorpora una serie de plugins que pueden ser usados en Nagios para crear comandos y servicios para comparar las métricas capturadas (o generadas) por Ganglia contra los umbrales de alerta definidos en Nagios.

Los cinco plug-ins de Ganglia son los siguientes:

- `check_heartbeat`: comprueba el heartbeat para verificar si el host está disponible.
- `check_metric`: comprueba una sola métrica en un host específico.
- `check_multiple_metrics`: comprueba múltiples métricas en un host específico.
- `check_host_regex`: comprueba múltiples métricas en un rango de hosts definido por regex.
- `check_value_same_everywhere`: comprueba que el valor o los valores son los mismos en un conjunto de hosts.



Para integrar Nagios con Ganglia es necesario asegurarse de que el proceso de Ganglia `gmond` se ejecuta en cada host que tiene un plug-in de Nagios en ejecución

Bibliografía

- [Koc14] Wojciech Kocjan. *Learning Nagios 4*. Packt Publishing Ltd, 2014.
- [Kuf16] Lukasz Kufel. Tools for distributed systems monitoring. *Foundations of Computing and Decision Sciences*, 41(4):237–260, 2016.
- [Lak14] Bhushan Lakhe. *Practical Hadoop Security*. Apress, USA, 1st edition, 2014.
- [MLN⁺12] Matt Massie, Bernard Li, Brad Nicholes, Vladimir Vuksan, Robert Alexander, Jeff Buchbinder, Frederiko Costa, Alex Dean, Dave Josephsen, Peter Phaal, et al. *Monitoring with Ganglia: tracking dynamic host and application metrics at scale*. .o'Reilly Media, Inc.", 2012.
- [OC14] Cristobal Ortega Carrasco. Diseño y evaluación de un clúster hpc: software de sistema. B.S. thesis, Universitat Politècnica de Catalunya, 2014.
- [WZL13] Mingli Wu, Zhongmei Zhang, and Yebai Li. Application research of hadoop resource monitoring system based on ganglia and nagios. In *2013 IEEE 4th International Conference on Software Engineering and Service Science*, pages 684–688. IEEE, 2013.